

v1.5 — Authentication, Authorization & Security Specification

Absolutely. Building directly on **v1.4 — REST API & OpenAPI Specification**, the next specification defines the security architecture that protects identities, organizations, cases, documents, APIs, and administrative functions.

ILCMS — Icelandic Legal Case Management System

Technical Specification v1.5

Authentication, Authorization & Security Specification

Project: Icelandic Legal Case Management System (ILCMS)

Version: 1.5

Status: Technical Specification

Target Market: Iceland

Primary Language: Icelandic

Architecture: Web application / API / worker-based backend

Backend: Python / FastAPI

Database: PostgreSQL

Deployment: Kubernetes / private cloud / on-premises / air-gapped capable

1. Purpose

This specification defines the authentication, authorization, security and security-management architecture for ILCMS.

The objective is to ensure that:

- only authenticated users can access protected functionality;
- users can access only organizations and cases for which they are authorized;
- access to legal documents is strictly controlled;
- privileged operations require appropriate permissions;
- tenant/organization boundaries cannot be bypassed;
- security-sensitive operations are auditable;
- secrets and cryptographic material are protected;
- the system remains secure in cloud, private-cloud, on-premises and air-gapped deployments;

- security controls are implemented consistently across the frontend, API, workers, database and object storage.

This specification builds upon:

- v1.1 — DevOps, CI/CD & Kubernetes
- v1.2 — Backend Implementation
- v1.3 — Database & Data Model
- v1.4 — REST API & OpenAPI

2. Security Principles

ILCMS shall follow these principles.

2.1 Zero Trust

No request is trusted solely because it originates from:

- an internal network;
- a Kubernetes namespace;
- an internal service;
- a VPN;
- an administrator workstation.

Every request must be authenticated and authorized according to its context.

2.2 Least Privilege

Users, services and administrators receive only the permissions required for their responsibilities.

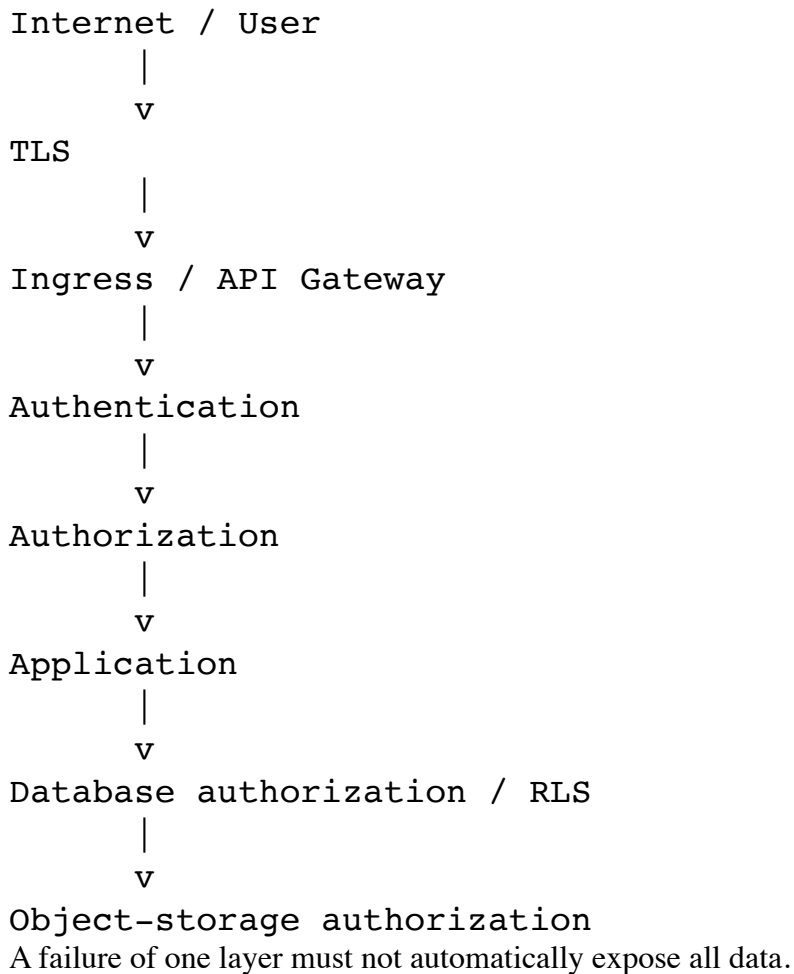
Permissions shall be:

- explicit;
- independently revocable;
- auditable;
- preferably time-limited for privileged access.

2.3 Defense in Depth

Security shall not depend upon a single control.

For example:



2.4 Secure by Default

Examples:

- new users receive no organizational access until assigned;
- new organizations contain no cross-organization access;
- documents are private by default;
- API endpoints require authentication unless explicitly public;
- audit logging is enabled by default;
- signed download URLs are short-lived;
- secrets are never returned through normal API responses.

2.5 Fail Closed

If authorization cannot be determined reliably, access shall be denied.

Examples:

Cannot determine organization membership
-> DENY

Cannot determine document ownership
-> DENY

Authorization service unavailable
-> DENY protected operation

The system must never interpret an authorization failure as permission.

3. Threat Model

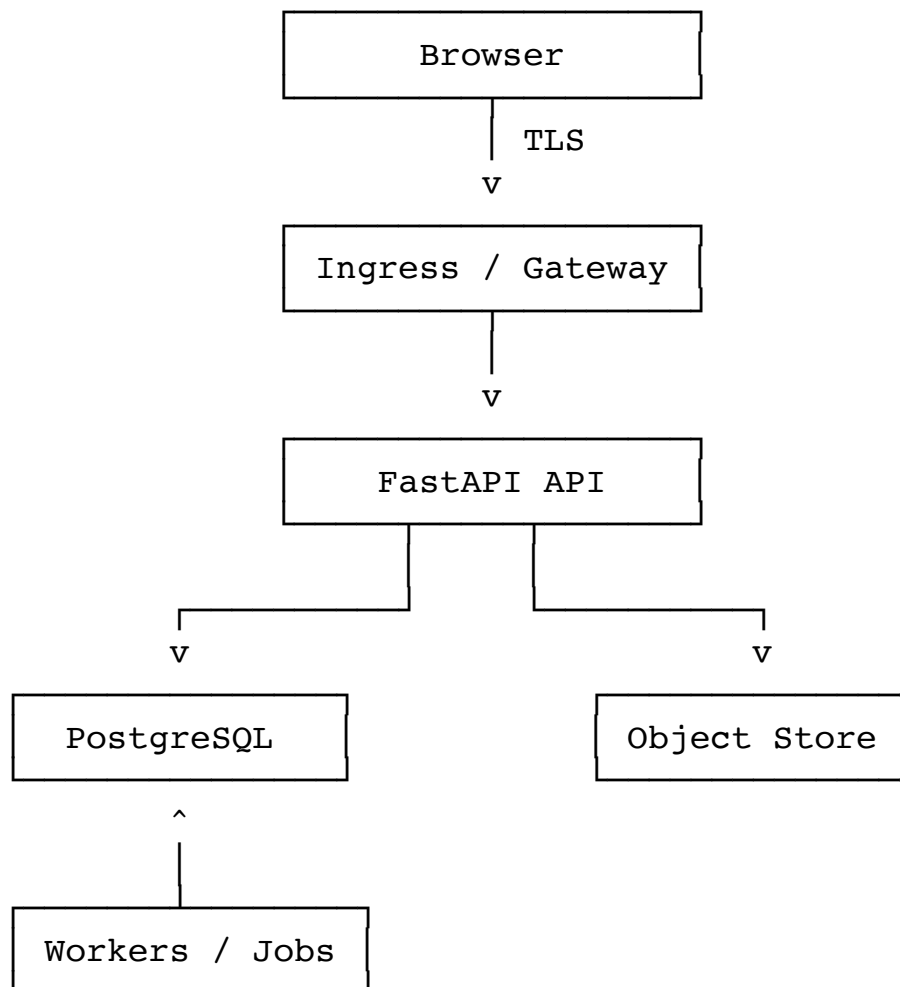
ILCMS shall consider at least the following threats.

Threat	Required mitigation
Stolen credentials	MFA, short-lived tokens, anomaly detection
Stolen access token	Short lifetime, TLS, revocation strategy
Session hijacking	Secure cookies/tokens, TLS, session controls
Privilege escalation	RBAC + object-level authorization
Cross-organization access	Tenant isolation + authorization + RLS
Document URL sharing	Short-lived signed URLs
Malicious document upload	Malware scanning and validation
SQL injection	Parameterized queries / ORM
XSS	Output encoding + CSP
CSRF	Appropriate token/cookie architecture
API abuse	Rate limiting
Brute-force login	Rate limiting + account protection
Insider misuse	Least privilege + audit
Administrator compromise	MFA + privileged access controls
Database compromise	Encryption + network isolation + RLS
Secret leakage	Vault + secret scanning
Supply-chain compromise	Dependency/container/SBOM scanning
Log leakage	PII/secrets filtering
Backup theft	Encryption + access control

Kubernetes compromise	Network policies + RBAC + workload isolation
-----------------------	--

4. Trust Boundaries

The following components are separate security boundaries:



Additional boundaries may exist around:

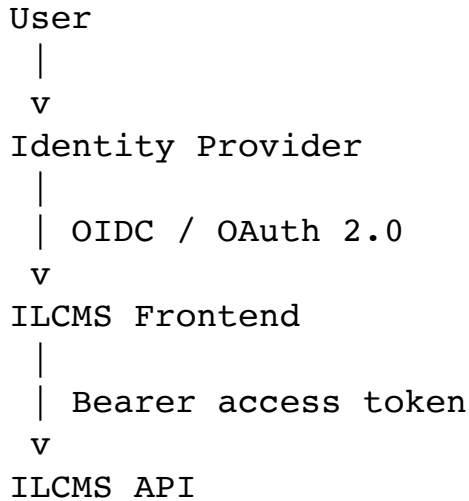
- Identity Provider;
- Vault;
- message broker;
- Redis;
- search engine;
- OCR service;
- AI services;
- external integrations.

Each integration must have its own authentication and authorization mechanism.

5. Identity Architecture

ILCMS shall preferably use an external Identity Provider (IdP) rather than implementing authentication itself.

The preferred architecture is:



The IdP should support:

- OpenID Connect;
- OAuth 2.0;
- MFA;
- account lifecycle management;
- group/role integration;
- session management;
- password policies where applicable;
- audit/security events.

Suitable deployment models may include:

- enterprise identity provider;
- private identity infrastructure;
- Keycloak;
- Microsoft Entra ID;
- another standards-compliant OIDC provider.

The ILCMS application must not depend on a specific provider.

An identity-provider adapter shall isolate provider-specific functionality.

6. OpenID Connect

ILCMS shall use OpenID Connect for user authentication.

The backend shall validate:

- token signature;
- issuer (**iss**);
- audience (**aud**);
- expiration (**exp**);
- issued-at (**iat**);
- subject (**sub**);
- nonce where applicable;
- authentication context where required.

The API must never trust identity information supplied only through frontend fields.

For example, this must not be sufficient:

```
{  
  "user_id": "123"  
}
```

The API must derive the authenticated identity from the validated security context.

7. JWT Access Tokens

Where JWT bearer tokens are used, the API shall validate them locally using trusted issuer metadata and signing keys.

A token may contain claims such as:

```
{  
  "iss": "https://identity.example",  
  "sub": "user-uuid",  
  "aud": "ilcms-api",  
  "exp": 1770000000,  
  "iat": 1769999700,  
}
```

```
"jti": "token-uuid",  
"email": "user@example.com",  
"name": "Example User"  
}
```

ILCMS-specific authorization data should not be placed unnecessarily into tokens.

In particular, rapidly changing permissions should preferably be retrieved from ILCMS rather than relying entirely on stale JWT claims.

8. Token Lifetime

Access tokens should be short-lived.

Recommended starting point:

Access token: 5–15 minutes
Refresh/session: longer-lived, controlled by IdP
Exact values shall be configurable.

Long-lived access tokens should not be used.

The application shall never store access tokens in:

- PostgreSQL;
- application logs;
- audit records;
- browser local storage where avoidable.

9. Refresh Tokens

Refresh tokens shall preferably remain under the control of the Identity Provider.

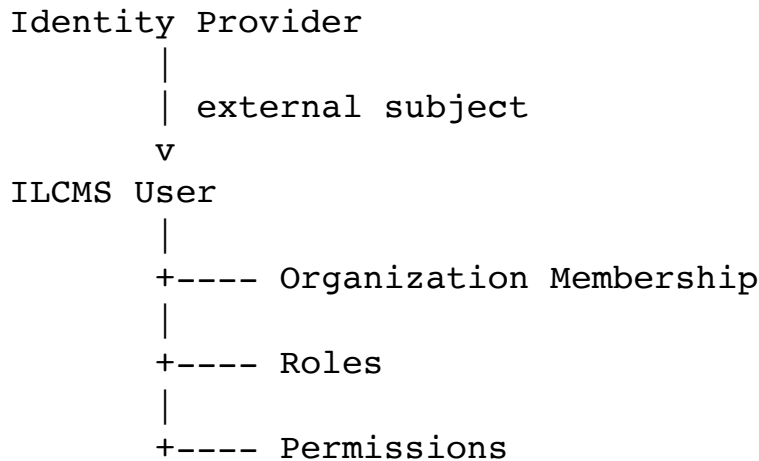
If browser-based authentication requires refresh tokens:

- use secure storage mechanisms;
- use refresh-token rotation where supported;
- revoke on logout or account compromise;
- never expose refresh tokens to backend logs;
- never include them in normal API responses.

10. User Provisioning

ILCMS shall maintain its own internal user record.

Conceptually:



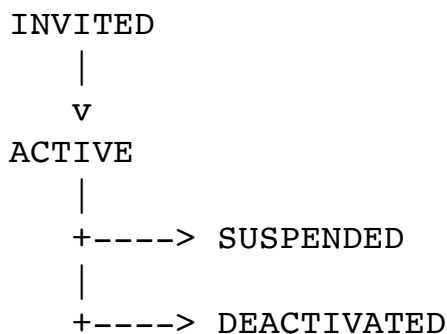
The ILCMS user should contain a stable external identity reference:

```
identity_provider
identity_subject
rather than relying on email as the permanent identifier.
```

Email addresses can change.

11. User Lifecycle

The following lifecycle shall be supported:



A deactivated user must immediately lose ILCMS access.

Historical audit events must remain associated with the user's immutable internal identity.

The system must not delete audit history merely because a user is deactivated.

12. Just-in-Time Provisioning

Optional JIT provisioning may create an ILCMS user when an authenticated identity first accesses the application.

However:

Authentication does not automatically grant organization access.

For example:

Successful login

|

v

Create ILCMS user

|

v

No organization membership

|

v

Access denied until administrator grants membership

This prevents accidental access to legal cases.

13. Organization / Tenant Security

The organization is the primary tenant boundary.

A user may belong to:

Organization A

Organization B

with different roles in each.

Example:

User X

Organization A:

LAWYER

Organization B:

READ_ONLY

Authorization must always be evaluated within the organization context.

14. Organization Context

Every organization-scoped API request shall have an unambiguous organization context.

Possible mechanisms include:

X-Organization-ID: <uuid>
or deriving the organization from the resource itself.

If a header is used, it must never be trusted merely because the user supplied it.

The backend must verify:

```
authenticated user
  |
  +--> member of requested organization?
        |
        YES  ---+--- NO
        |      |
        CONTINUE  DENY
```

15. Tenant Isolation

Tenant isolation shall exist at multiple levels.

Application level

Every query must be scoped to the authorized organization.

Service level

Workers must receive organization context from trusted internal messages.

Database level

PostgreSQL Row-Level Security (RLS) should be considered for high-value organization-scoped tables.

Object-storage level

Documents must use organization-aware storage authorization.

The goal is to prevent a programming error such as:

```
SELECT * FROM cases;
```

from accidentally exposing another organization's cases.

16. PostgreSQL Row-Level Security

Where feasible, PostgreSQL RLS shall provide a second authorization boundary.

Conceptually:

```
SET app.organization_id = '...';
```

```
SELECT *
```

```
FROM cases;
```

RLS policies then restrict visible rows.

The application database role should not have unrestricted access to protected tables unless explicitly required for administrative operations.

Care must be taken with:

- migrations;
- background workers;
- reporting;
- database administration;
- backups;
- superuser roles.

RLS is a defense-in-depth mechanism and does not replace application authorization.

17. Role-Based Access Control

ILCMS shall use RBAC as the primary authorization model.

Example roles:

SYSTEM_ADMIN

ORGANIZATION_ADMIN

LAWYER

CASE_MANAGER

ASSISTANT

READ_ONLY

AUDITOR

The actual role set should remain configurable.

Roles represent collections of permissions.

18. Permission Model

Permissions should be expressed as stable machine-readable identifiers.

Examples:

case:create
case:read
case:update
case:delete

case:assign
case:close
case:reopen

document:create
document:read
document:update
document:delete
document:download

task:create
task:update

user:read
user:create
user:update

organization:admin

audit:read
export:create

The frontend may use these permissions to control UI visibility, but the backend remains authoritative.

19. Object-Level Authorization

RBAC alone is insufficient.

For example:

LAWYER

does not necessarily mean:

LAWYER -> every case in organization

A user may be authorized only for:

- assigned cases;
- cases belonging to a department;

- cases they have been explicitly granted access to;
- cases belonging to a permitted client.

Therefore ILCMS shall support object-level authorization.

Example:

```
User
|
+-- Organization membership
|
+-- Role
|
+-- Case access
    |
    +-- Owner
    +-- Assigned lawyer
    +-- Team member
    +-- Explicit permission
```

20. Authorization Decision

The backend should conceptually perform:

```
authorize(
    user=current_user,
    action="document:read",
    resource=document,
    organization=organization,
)
```

The authorization service should evaluate:

1. authentication;
2. account status;
3. organization membership;
4. role;
5. permission;
6. resource ownership/access;
7. additional restrictions.

Only then should the operation proceed.

21. Attribute-Based Controls

Future versions may support ABAC-style conditions.

Examples:

```
department == case.department  
or:
```

```
user.is_external == false  
or:
```

```
document.classification <= user.max_classification
```

ABAC should be introduced only where justified because excessive authorization complexity can itself become a security risk.

22. Case Access

Cases should support explicit access controls.

Potential model:

```
Case  
|  
+-- Organization  
|  
+-- Owner  
|  
+-- Assigned users  
|  
+-- Assigned teams  
|  
+-- Explicit access grants
```

Access grants should support at least:

READ

WRITE

ADMIN

or equivalent permission sets.

23. Document Security

Documents are particularly sensitive in a legal case-management system.

A document must inherit or explicitly define authorization associated with its case.

Users must not be able to obtain a document merely by guessing its UUID.

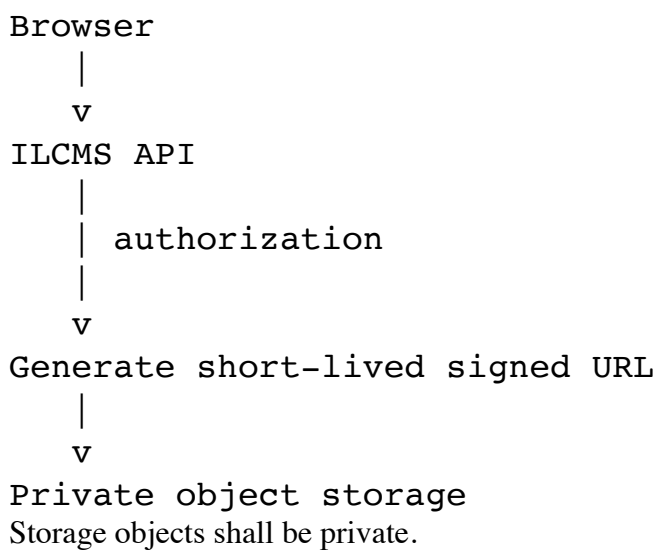
This must be rejected:

GET /documents/<known-or-guessed-id>
unless the authenticated user has permission to access that document.

24. Object Storage

Documents should not normally be served directly from public object storage.

Recommended architecture:



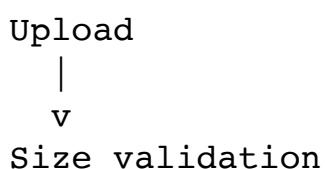
Signed URLs should:

- expire quickly;
- be generated only after authorization;
- be restricted to the intended object;
- never be permanently stored in the database.

25. Document Upload Security

Uploads shall pass through a security pipeline.

Recommended:



|
v
MIME/type validation
|
v
Checksum
|
v
Malware scan
|
v
File validation
|
v
Metadata extraction
|
v
OCR / text extraction
|
v
Indexing

Potentially dangerous files must not become available to users before security processing is complete.

26. File Type Validation

ILCMS must not trust:

Content-Type
provided by the browser.

Validation should consider:

- file signature/magic bytes;
- MIME type;
- extension;
- parser behavior;
- configured allowed formats.

Allowed formats should be explicitly configured.

27. Malware Protection

Uploaded files should be scanned using a malware scanning engine.

The result must be persisted as processing metadata.

Example:

```
virus_scan:
    status = PASSED
    scanner = clamav
    scanner_version = ...
    scanned_at = ...
```

Files that fail scanning shall not be downloadable.

28. Multi-Factor Authentication

MFA shall be strongly recommended and preferably mandatory for:

- system administrators;
- organization administrators;
- users with access to highly sensitive cases;
- export functionality;
- security configuration;
- break-glass operations.

Supported mechanisms may include:

- FIDO2/WebAuthn;
- passkeys;
- TOTP;
- enterprise IdP MFA.

SMS should not be the preferred MFA mechanism where stronger methods are available.

29. Local Authentication

Local username/password authentication should not be implemented unless required.

If required, passwords must be:

- strongly hashed using Argon2id or equivalent;

- salted;
- never reversible;
- never logged;
- never stored in plaintext.

The application must not implement custom cryptography for password storage.

30. Password Requirements

If local authentication exists, password policy should include:

- minimum length;
- compromised-password detection;
- rate limiting;
- login attempt protection;
- secure password reset;
- MFA support.

Forced periodic password changes should not be used as the primary security control unless required by an applicable policy.

31. Browser Session Security

For cookie-based authentication:

`Secure`
`HttpOnly`
`SameSite=Lax/Strict`
should be used where appropriate.

Session cookies must not be accessible to JavaScript.

The application shall prevent session fixation.

32. CSRF

If authentication uses cookies, CSRF protection is mandatory for state-changing operations.

If authentication uses bearer tokens that are not automatically attached by the browser, CSRF exposure is substantially different, but other browser security controls remain necessary.

The final authentication architecture must explicitly document which model is used.

33. CORS

CORS shall use an explicit allowlist.

Production must not use:

`Access-Control-Allow-Origin: *`
for authenticated APIs.

Allowed origins should be environment-specific.

34. Security Headers

The frontend/API gateway should provide appropriate security headers, including as applicable:

`Strict-Transport-Security`

`Content-Security-Policy`

`X-Content-Type-Options`

`Referrer-Policy`

`Permissions-Policy`

Clickjacking protection should also be implemented where applicable.

35. TLS

All production communication must use TLS.

This includes:

- browser → ingress;
- API → database;
- API → object storage;
- API → message broker;
- API → Redis;
- services → IdP;
- services → Vault;

- worker → external services.

Plaintext credentials or secrets must never traverse production networks.

36. Internal Service Authentication

Services should not automatically trust requests from another Kubernetes pod.

For sensitive service-to-service operations, use:

- workload identity;
- mTLS;
- OAuth2 client credentials;
- signed internal requests;
- Kubernetes-native identity mechanisms.

The selected mechanism should be standardized rather than implemented separately by every service.

37. Service Accounts

Background workers and services shall use dedicated identities.

Example:

```
ilcms-api
ilcms-worker
ilcms-scheduler
ilcms-document-worker
ilcms-search-worker
```

Each identity receives only the permissions required by its function.

The document worker should not automatically have:

```
user administration
permissions.
```

38. API Keys

API keys may be supported for controlled integrations.

Requirements:

- generated cryptographically securely;
- shown only once;
- stored hashed;
- scoped;
- associated with an organization/service;
- expiration supported;
- revocation supported;
- last-used timestamp recorded;
- audited.

Example:

Integration:
 accounting-system

Permissions:
 case:read
 document:read

Expires:
 2027-01-01

API keys must never have unrestricted administrator privileges by default.

39. Secrets Management

Secrets shall not be stored in:

- Git;
- Dockerfiles;
- Helm values committed to Git;
- source code;
- Kubernetes manifests in plaintext;
- CI logs;
- application logs.

Secrets include:

- database credentials;
- signing keys;
- API credentials;
- object-storage credentials;
- IdP secrets;
- encryption keys;
- integration credentials.

40. Vault

The architecture should use Vault or an equivalent secrets-management system.

Example:

```
Vault
|
+-- database credentials
+-- IdP credentials
+-- storage credentials
+-- integration secrets
+-- encryption material
```

Kubernetes workloads should retrieve secrets through controlled mechanisms such as:

- External Secrets;
- Vault Agent;
- CSI integration;
- workload identity.

Secrets should be rotated.

41. Secret Rotation

Secrets must have documented rotation procedures.

Rotation must be possible without requiring a full application rebuild.

High-priority secrets include:

- database credentials;

- service credentials;
- signing keys;
- integration credentials.

Where possible, applications should support overlapping credentials during rotation.

42. Encryption at Rest

The following should be encrypted at rest:

- PostgreSQL storage;
- document/object storage;
- backups;
- Kubernetes persistent volumes where supported;
- secret stores.

Encryption keys must be managed separately from the encrypted data.

43. Application-Level Encryption

Application-level encryption should be considered for particularly sensitive fields.

Examples might include:

- highly sensitive personal information;
- credentials;
- special confidential data;
- selected integration secrets.

However, application-level encryption should not be applied indiscriminately because it can make:

- searching;
- indexing;
- reporting;
- migrations

more difficult.

44. Cryptographic Key Management

Encryption keys must not be embedded in application source code.

Key management should support:

- secure generation;
- access control;
- rotation;
- versioning;
- revocation;
- backup/recovery procedures.

Key identifiers should be stored alongside encrypted data when required for key rotation.

45. Audit Logging

ILCMS shall maintain an immutable audit trail for security-relevant operations.

At minimum, audit events should include:

Authentication

Authorization changes

User creation/deactivation

Role changes

Organization membership changes

Case creation/deletion

Case access changes

Document upload

Document download

Document deletion

Document sharing

Exports

Administrative actions

Security configuration changes

API key creation/revocation

Break-glass access

46. Audit Event Structure

Example:

```
{
  "event_id": "uuid",
  "timestamp": "2026-09-02T08:00:00Z",
  "actor_user_id": "uuid",
  "organization_id": "uuid",
  "action": "document.download",
  "resource_type": "document",
  "resource_id": "uuid",
  "result": "success",
  "request_id": "uuid",
  "source_ip": "...",
  "metadata": {}
}
```

Audit records must not contain:

- passwords;
- access tokens;
- refresh tokens;
- secret values;
- private encryption keys.

47. Security Events vs Application Logs

These should remain conceptually separate.

Application logs

Used for:

- debugging;
- operations;
- performance;
- errors.

Audit events

Used for:

- accountability;

- legal/security review;
- administrative history;
- access tracking.

Audit events should have stronger retention and integrity requirements.

48. Log Protection

Logs must be protected against:

- unauthorized access;
- modification;
- accidental secret disclosure.

Sensitive values must be redacted.

For example:

`Authorization: Bearer eyJ...`
must never appear in logs.

Instead:

`Authorization: [REDACTED]`

49. Request Correlation

Every API request should receive a request ID.

Example:

`X-Request-ID: 6d9e...`
The identifier should connect:

```
API request
|
+-- application log
|
+-- audit event
|
+-- worker job
|
+-- error report
```

This is particularly important when investigating access to legal documents.

50. Rate Limiting

Rate limiting shall be applied to:

- authentication;
- password reset;
- API endpoints;
- search;
- document uploads;
- document processing;
- exports;
- AI endpoints;
- administrative operations.

Limits should be configurable by:

- IP;
- user;
- organization;
- API key;
- endpoint.

51. Brute-Force Protection

Authentication infrastructure should provide protection against:

- password guessing;
- credential stuffing;
- automated login attempts;
- password-reset abuse.

Possible controls include:

- rate limiting;

- temporary throttling;
- MFA;
- breached-password detection;
- IdP-native protection.

The system should avoid revealing whether a particular account exists.

52. Sensitive Resource Enumeration

APIs should avoid leaking resource existence.

For example, where appropriate:

User without access to case 123
should receive:

404 Not Found
rather than:

403 Forbidden
if the distinction itself reveals sensitive information.

The decision should be consistent throughout the API.

53. Search Authorization

Search must never become an authorization bypass.

For example:

GET /api/v1/search?q=confidential
must return only documents/cases/parties the user is authorized to see.

Authorization filtering must occur server-side.

It must not rely on the frontend hiding unauthorized results.

54. Export Security

Exports are high-risk operations.

Export permissions should be separate from normal read permissions where appropriate.

An export should record:

who
what
when
organization
resource scope
format
reason, if required
Exports should be:

- asynchronous;
- audited;
- access-controlled;
- encrypted;
- automatically expired;
- securely deleted after expiration.

55. Administrative Access

Administrative operations require elevated permissions.

Examples:

SYSTEM_ADMIN

ORGANIZATION_ADMIN

must not automatically imply access to all legal documents unless explicitly designed and authorized.

Technical administration and legal-data access should preferably be separated.

For example:

Infrastructure Administrator

!=

Legal Case Administrator

56. Privileged Access

Privileged users should use:

- MFA;
- dedicated accounts where practical;
- stronger session controls;

- extensive audit logging;
- limited administrative access;
- periodic access review.

Administrative accounts should not be used for ordinary case work.

57. Break-Glass Access

ILCMS may implement emergency access for exceptional circumstances.

Example:

Normal authorization

|

v

Access denied

Emergency authorization

|

v

Break-glass

|

v

Temporary elevated access

Break-glass access must:

- require strong authentication;
- require justification;
- be time-limited;
- generate a high-priority audit event;
- notify designated administrators where appropriate;
- be reviewed afterwards.

It must never become a normal authorization mechanism.

58. Database Security

PostgreSQL shall be isolated from public networks.

Only approved application/service identities should access it.

Database access shall use:

- TLS where appropriate;
- strong credentials;
- network restrictions;
- least privilege;
- separate administrative accounts.

The API should not use PostgreSQL superuser credentials.

59. Database Administrative Access

Database administrators should use separate credentials from application identities.

Production database access should be:

- restricted;
- audited;
- preferably time-limited;
- MFA-protected through the access mechanism where available.

60. Kubernetes Security

Kubernetes security shall include:

- namespace isolation;
- RBAC;
- NetworkPolicies;
- Pod Security controls;
- non-root containers where possible;
- read-only filesystems where possible;
- dropped Linux capabilities;
- resource limits;
- image provenance/scanning;

- restricted service accounts.

Applications must not run with unnecessary Kubernetes privileges.

61. Container Security

Containers should:

- use minimal base images;
- run as non-root;
- avoid unnecessary packages;
- have pinned dependencies;
- be scanned for vulnerabilities;
- generate SBOMs;
- be rebuilt regularly.

Container images should preferably be immutable and referenced by digest in production.

62. Supply Chain Security

CI/CD shall include:

Source

```
|
+-- SAST
+-- dependency scan
+-- secret scan
+-- IaC scan
+-- container scan
+-- SBOM
```

|

v

Build

|

v

Signed artifact

|

v

Private registry

Production deployment should only accept approved artifacts.

63. Dependency Security

Python and frontend dependencies shall be:

- pinned or constrained;
- scanned for known vulnerabilities;
- updated regularly;
- reviewed for abandoned/high-risk packages.

Critical vulnerabilities should trigger expedited remediation.

64. API Security

All protected API endpoints shall require authentication and authorization.

Security checks must occur before:

- database retrieval;
- document access;
- expensive processing;
- external integration calls.

This prevents unauthorized users from using expensive operations as a resource-exhaustion mechanism.

65. Input Validation

All API inputs shall be validated using strongly typed schemas.

Validation includes:

- type;
- length;
- range;
- allowed enum values;
- file size;
- file format;

- identifiers;
- pagination limits.

User-controlled strings must never be concatenated into SQL or shell commands.

66. Command Execution

If the backend or worker executes operating-system commands, inputs must never be directly concatenated into shell commands.

Prefer:

```
subprocess.run(  
    ["command", "--option", validated_value],  
    check=True,  
)
```

rather than shell string construction.

67. SSRF Protection

Any feature that retrieves a URL supplied by a user must implement SSRF protection.

Examples include:

- document import from URL;
- webhook testing;
- integration configuration;
- remote document retrieval.

Requests to internal/private addresses must be restricted unless explicitly required.

68. External Integrations

External integrations shall use separate credentials and permissions.

Each integration should have:

Integration identity
Credential
Scope
Allowed endpoints
Expiration

Audit events

External systems must never receive unrestricted ILCMS credentials.

69. AI Security Boundary

If AI functionality is introduced, it shall be treated as a separate security boundary.

The AI subsystem must not automatically receive all case data.

Instead:

User

|

v

ILCMS authorization

|

v

Allowed case/document context

|

v

AI subsystem

AI operations should record:

- user;
- organization;
- case;
- model;
- model version;
- prompt/template version;
- timestamp;
- operation type.

External AI providers must not receive legal data unless explicitly configured and permitted.

Local/private AI deployments should be supported for sensitive environments.

70. Air-Gapped Security

The core security architecture must support fully disconnected environments.

An air-gapped deployment should be capable of operating with:

Internal IdP

Internal PostgreSQL

Internal object storage

Internal message broker

Internal OCR

Internal search

Internal AI

Internal Vault

No core authentication or authorization functionality should require an Internet connection.

External services must be optional adapters.

71. Backup Security

Backups must be:

- encrypted;
- access-controlled;
- monitored;
- protected against accidental deletion;
- tested through restoration.

Backups must inherit appropriate confidentiality controls because they may contain complete legal cases.

Backup credentials must be separate from ordinary application credentials.

72. Backup Deletion Protection

Where infrastructure supports it, backups should use:

- immutability;
- retention locks;
- write-once mechanisms;
- separate administrative credentials.

This reduces the impact of ransomware or administrator compromise.

73. Security Monitoring

Security monitoring should detect:

- repeated failed logins;
- unusual login locations;
- privilege escalation;
- mass document downloads;
- unusual exports;
- repeated authorization failures;
- API abuse;
- disabled security controls;
- break-glass access;
- unexpected service identities.

Monitoring thresholds should be configurable.

74. Anomaly Detection

Future versions may introduce behavioral detection.

Potential signals:

Normal:

5 document downloads/day

Abnormal:

5,000 document downloads/hour

Such systems should generate alerts rather than automatically blocking users unless confidence is sufficiently high.

75. Access Reviews

Organization administrators should periodically review:

- users;
- organization memberships;
- roles;
- case assignments;

- service accounts;
- API keys.

High-risk permissions should receive more frequent review.

76. Data Minimization

ILCMS should collect only information required for:

- legal case management;
- security;
- auditing;
- operational requirements;
- configured integrations.

Sensitive data should not be copied unnecessarily between services.

77. Privacy-Aware Logging

The following must generally not appear in ordinary logs:

- document contents;
- legal correspondence;
- passwords;
- authentication tokens;
- complete personal records;
- unnecessary personal identifiers.

Where an identifier is required for troubleshooting, use internal IDs rather than copying entire records.

78. Error Handling

Security errors must not disclose implementation details.

Do not return:

PostgreSQL exception:

relation public.secret_table does not exist
Return:

```
{
  "error": {
    "code": "INTERNAL_ERROR",
    "message": "An internal error occurred.",
    "request_id": "...",
  }
}
```

Detailed information belongs in protected server-side logs.

79. Security Configuration

Security configuration must be environment-specific.

Examples:

```
AUTH_ISSUER
AUTH_AUDIENCE
ACCESS_TOKEN_TTL
SESSION_TIMEOUT
MFA_REQUIRED
RATE_LIMITS
CORS_ALLOWED_ORIGINS
MAX_UPLOAD_SIZE
ALLOWED_FILE_TYPES
SIGNED_URL_TTL
```

Secrets must not be confused with ordinary configuration.

80. Security Defaults

Recommended defaults:

Authentication:	Required
MFA:	Required for privileged users
Document storage:	Private
Signed URL lifetime:	Short
Audit logging:	Enabled
Rate limiting:	Enabled
CORS:	Allowlist
Database:	Private
Object storage:	Private
Containers:	Non-root

Secrets:	Vault-managed
TLS:	Required
API version:	/api/v1

81. Authorization Architecture

The backend should implement a central authorization layer rather than scattering authorization logic throughout endpoint implementations.

Conceptually:

```
FastAPI Endpoint
    |
    v
Authentication dependency
    |
    v
Authorization service
    |
    +-- Organization membership
    +-- Role
    +-- Permission
    +-- Resource access
    +-- Additional policy
    |
    v
```

Domain operation

This reduces inconsistent authorization behavior.

82. Example FastAPI Pattern

Conceptually:

```
@router.get("/cases/{case_id}")
async def get_case(
    case: Case = Depends(get_case),
    current_user: User = Depends(get_current_user),
):
    authorize(
        current_user,
        "case:read",
        case,
    )
```

`return case`

The actual implementation should centralize policy evaluation rather than duplicating complex authorization logic in every endpoint.

83. Authorization Testing

Every protected resource shall have tests covering:

Authenticated + authorized

Authenticated + unauthorized

Unauthenticated

Wrong organization

Wrong case

Suspended user

Deactivated user

Expired token

Invalid token

Missing permission

Administrator

Service account

Cross-tenant authorization tests are mandatory.

84. Security Test Matrix

The CI pipeline should include:

Static

- SAST;
- dependency scanning;
- secret scanning;
- IaC scanning.

Application

- authentication tests;
- authorization tests;
- input validation;
- API security tests.

Infrastructure

- container scanning;
- Kubernetes configuration scanning;
- TLS configuration testing.

Dynamic

- DAST;
- API penetration testing;
- authenticated security testing.

85. Penetration Testing

Before production release, ILCMS should undergo independent penetration testing.

Testing should specifically include:

- authentication;
- authorization;
- tenant isolation;
- document access;
- file uploads;
- API security;
- search;
- exports;
- administrative functions;
- Kubernetes exposure;
- object storage;
- SSRF;
- XSS;
- CSRF;
- injection attacks.

High-severity findings must be resolved before production deployment.

86. Threat Scenarios

Scenario 1 — User guesses another document UUID

```
User
|
v
GET /documents/<UUID>
|
v
Authorization check
|
+-- Not authorized
|
v
404
No document metadata is returned.
```

Scenario 2 — User changes organization header

```
X-Organization-ID: another-org
Backend checks membership.

Membership = false
|
v
403/404
The header alone cannot change authorization.
```

Scenario 3 — Stolen signed URL

A signed URL should have a short expiration.

Example:

```
Issued:    10:00
Expires:   10:05
After expiration, the URL is useless.
```

Where supported, additional object/storage restrictions should be used.

Scenario 4 — Compromised worker

A document-processing worker should not have:

```
user:admin
organization:admin
permissions.
```

Its identity should be restricted to:

```
document:process
document:read-temporary
document:update-processing-status
This limits the damage from worker compromise.
```

Scenario 5 — Database query bug

Application authorization accidentally fails.

PostgreSQL RLS provides a second protection layer and prevents access to rows outside the current organization context.

87. Security Incident Response

The system must support investigation of:

```
Who?
What?
When?
Where?
Which organization?
Which resource?
Which operation?
What was the result?
Incident responders should be able to correlate:
```

```
request_id
user_id
organization_id
audit_event_id
job_id
timestamp
without exposing case contents unnecessarily.
```

88. Account Compromise Procedure

The operational runbook should support:

- 1. Disable user.
- 2. Revoke active sessions through IdP.
- 3. Revoke API keys.
- 4. Review recent authentication.
- 5. Review recent document access.
- 6. Review exports.
- 7. Review administrative operations.
- 8. Identify affected cases/documents.
- 9. Preserve audit evidence.
- 10. Re-enable only after investigation.

89. Security Event Severity

Suggested levels:

INFO

WARNING

HIGH

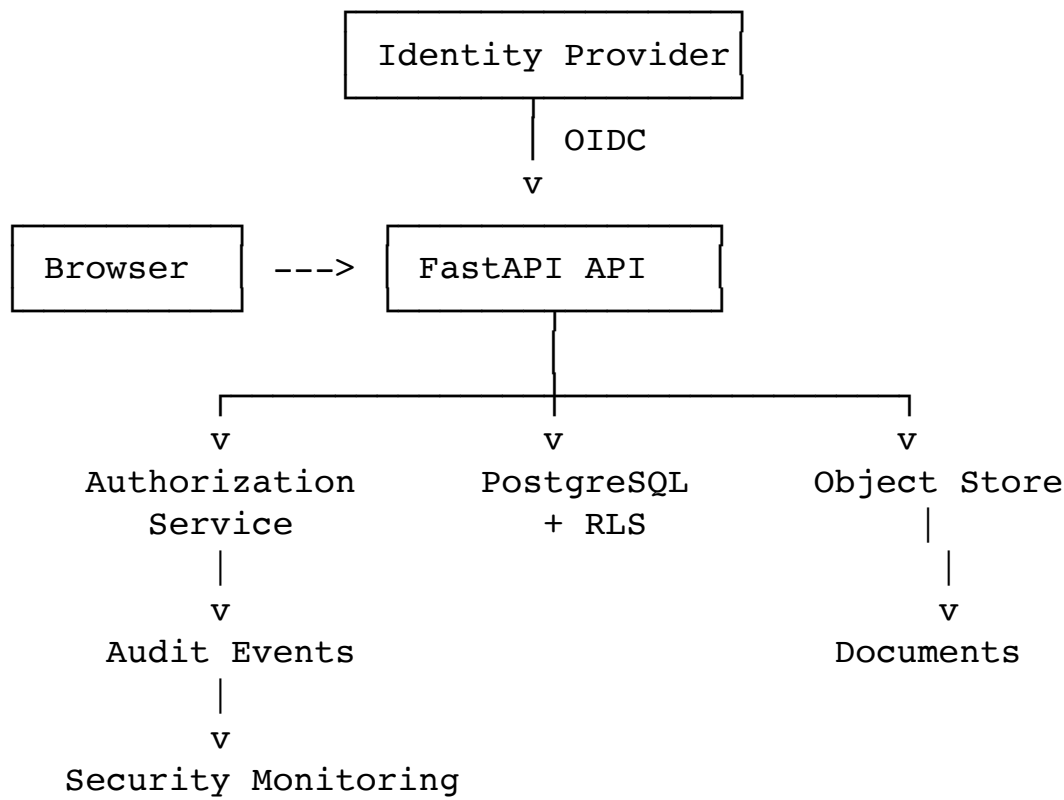
CRITICAL

Examples:

Event	Severity
Failed login	INFO/ WARNING
Repeated failed logins	WARNING
Privilege change	HIGH
Break-glass access	HIGH
Mass document download	HIGH
Audit subsystem disabled	CRITICAL
Suspected tenant isolation failure	CRITICAL

90. Security Architecture Components

The ILCMS security architecture should ultimately contain:



Additional:

Vault
Kubernetes RBAC
NetworkPolicies
Message Broker
Workers
Search
OCR
AI

91. Recommended Initial RBAC

A reasonable MVP role model is:

Role	Purpose
SYSTEM_ADMIN	Platform/system administration
ORGANIZATION_ADMIN	Organization administration
LAWYER	Normal legal case work

CASE_MANAGER	Case coordination/ administration
ASSISTANT	Supporting case work
READ_ONLY	Read-only access
AUDITOR	Restricted audit/reporting access

The exact permissions must be configurable.

92. Recommended Permission Groups

Organization

```
organization:read
organization:update
organization:admin
```

Users

```
user:read
user:create
user:update
user:disable
role:assign
```

Cases

```
case:create
case:read
case:update
case:delete
case:assign
case:close
case:reopen
```

Documents

```
document:create
document:read
document:update
document:download
document:delete
```

Tasks

```
task:create
task:read
task:update
task:complete
```

Audit

`audit:read`

Exports

`export:create`

`export:download`

Administration

`security:admin`

`system:admin`

93. Security Data Model

The v1.3 database model should support at least:

`users`

`organizations`

`organization_memberships`

`roles`

`permissions`

`role_permissions`

`user_roles`

`case_access`

`document_access`

`api_keys`

`sessions / external_identity_links`

`audit_events`

`security_events`

Where possible, authorization relationships should use immutable UUID identifiers.

94. Audit Immutability

Audit events should be append-only.

Application users must not be able to:

`UPDATE audit_events`

`DELETE audit_events`

Audit storage should preferably provide additional tamper resistance through:

- restricted database permissions;
- append-only mechanisms;

- separate storage;
- immutable object storage;
- cryptographic integrity mechanisms where required.

95. Security and Legal/Regulatory Requirements

ILCMS security requirements must be designed to support applicable Icelandic and European requirements where relevant.

However, this specification does **not** constitute legal advice or a legal compliance certification.

The final production system should be reviewed against the specific obligations applicable to:

- the operating organization;
- its role as data controller/processor;
- legal-sector confidentiality obligations;
- records/archival obligations;
- contractual requirements;
- applicable Icelandic legislation;
- applicable EEA/EU requirements.

Legal and archival assumptions should remain configurable rather than hard-coded into security logic.

96. Security Configuration Must Be Auditable

Changes to security configuration shall generate audit events.

Examples:

```
MFA requirement changed
Role modified
Permission modified
API rate limit changed
Organization security policy changed
External IdP changed
Encryption configuration changed
```

97. Security Administration API

The following administrative APIs should be considered:

GET /api/v1/security/policies

PATCH /api/v1/security/policies

GET /api/v1/roles

POST /api/v1/roles

GET /api/v1/permissions

GET /api/v1/users/{id}/roles

POST /api/v1/users/{id}/roles

DELETE /api/v1/users/{id}/roles

GET /api/v1/security/events

All must have strict administrative authorization.

98. Production Security Checklist

Before production:

Identity

- OIDC configured
- Issuer validation enabled
- Audience validation enabled
- Token expiry validated
- MFA enabled for privileged users
- User deactivation tested

Authorization

- RBAC implemented
- Object-level authorization implemented
- Organization isolation tested
- Cross-tenant tests implemented
- Search authorization implemented

- Document authorization implemented

Data

- PostgreSQL private
- RLS evaluated/implemented
- Object storage private
- Encryption enabled
- Backups encrypted

Secrets

- Vault configured
- No secrets in Git
- No secrets in container images
- Rotation procedure documented

Application

- Input validation
- Rate limiting
- CSRF protection where applicable
- CORS allowlist
- Security headers
- SSRF protections

Infrastructure

- Containers non-root
- NetworkPolicies
- Kubernetes RBAC
- Image scanning
- Dependency scanning
- SBOM

Monitoring

- Audit logging
- Security events
- Centralized logs
- Alerting
- Incident-response procedures

99. Definition of Done

v1.5 is considered implemented when:

1. OIDC authentication works.
2. Access tokens are correctly validated.
3. User identity is derived from authenticated context.
4. Organization membership is enforced.
5. RBAC is implemented.
6. Object-level authorization is implemented.
7. Cross-tenant access tests pass.
8. Document authorization is enforced.
9. Search authorization is enforced.
10. PostgreSQL security controls are implemented.
11. Secrets are managed through Vault or equivalent.
12. MFA is enforced for privileged accounts.
13. Audit events are generated for security-sensitive actions.
14. Logs do not contain credentials or tokens.
15. Rate limiting is active.
16. File-upload security controls are active.
17. Object storage is private.
18. Signed download URLs are short-lived.
19. Security headers and CORS are configured.
20. Kubernetes workloads use least privilege.

21. Security tests execute in CI.
22. Backup security has been validated.
23. Incident-response procedures exist.
24. Penetration testing has been completed before production.
25. Air-gapped deployment does not require Internet-based authentication or core security services.

100. Implementation Priority

Phase 1 — MVP Security Foundation

Implement first:

OIDC

↓

Authentication

↓

Organization membership

↓

RBAC

↓

Object authorization

↓

Audit

Plus:

- TLS;
- Vault;
- private object storage;
- secure document downloads;
- rate limiting;
- security logging.

Phase 2 — Defense in Depth

Add:

- PostgreSQL RLS;

- MFA enforcement;
- advanced security monitoring;
- API keys;
- security event detection;
- backup immutability;
- stronger Kubernetes controls.

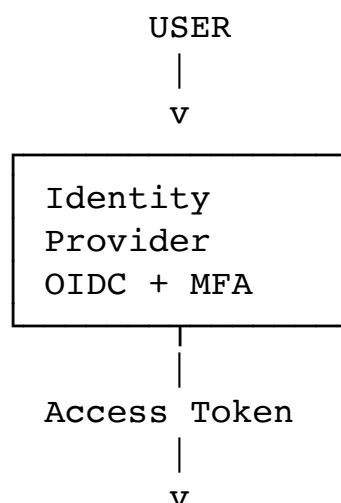
Phase 3 — Advanced Security

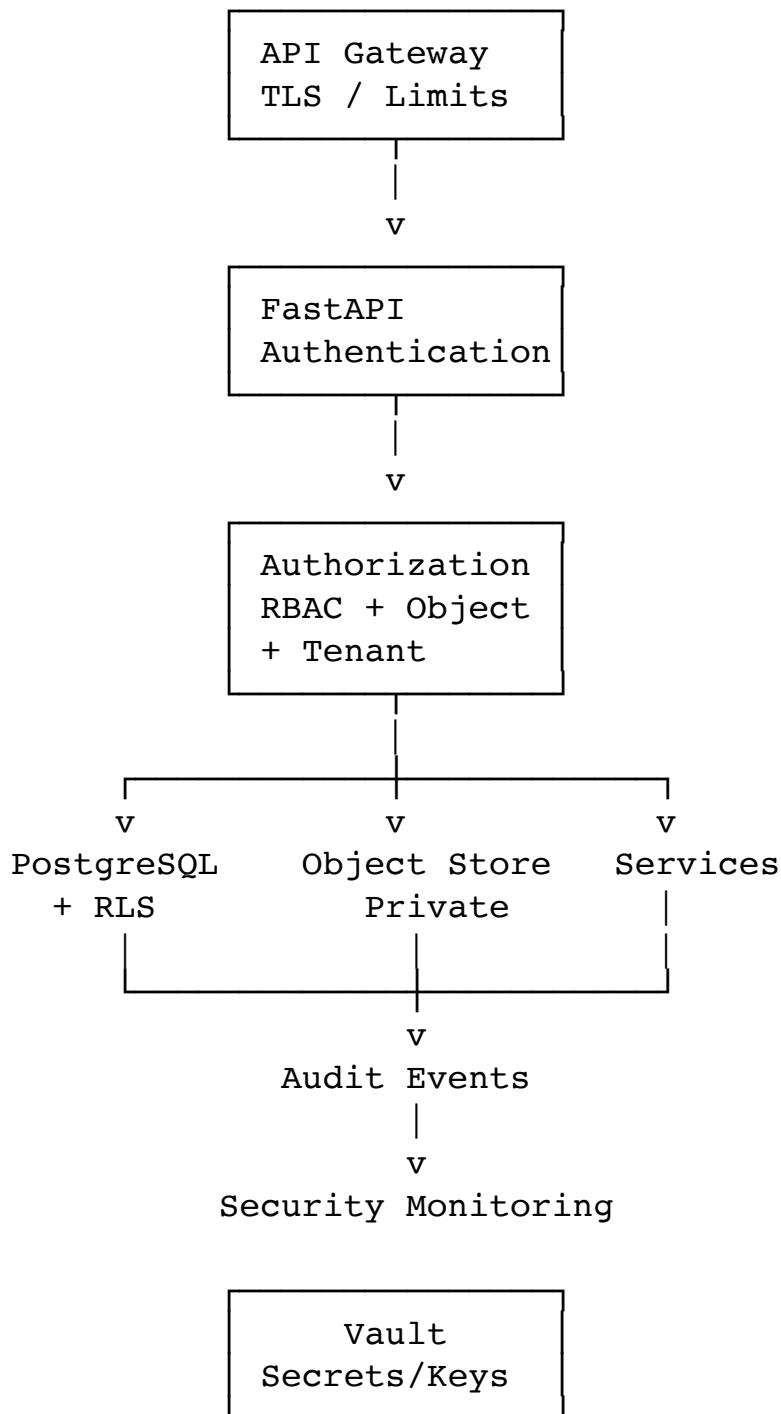
Potential additions:

- WebAuthn/passkeys;
- ABAC;
- anomaly detection;
- privileged access management;
- cryptographic audit integrity;
- automated access reviews;
- advanced DLP;
- confidential-computing options where justified.

101. Final Security Architecture

The resulting security model should be:





The fundamental security rule for ILCMS is:

Authentication establishes who the user is. Authorization determines what that user may access. Database and infrastructure controls provide additional protection if application-level controls fail.

This separation should remain fundamental throughout the implementation.

This gives us a solid security foundation for the system. **The next logical specification is v1.6 — Document Management, Electronic Records & Archiving Specification**, where we can go considerably deeper into document versions, Icelandic court-document conventions, OCR, metadata, retention, archival classifications, immutable records, document lifecycle, and integration with the Icelandic archival requirements.

